
HoT3D Ego Hand Tracking Research Report

Jake Digiugno

Department of Computer Science
Binghamton University
4400 Vestal Parkway East, Vestal, NY 13850

Abstract

This project implements and evaluates baseline methods for ego-centric hand pose estimation using the HoT3D dataset. A PyTorch-based pipeline is developed to load synchronized multi-view image data and corresponding ground-truth hand pose annotations from Project Aria recordings. Three baseline models are evaluated: a single-view model, a multi-view average fusion model, and a multi-view concatenation model. All models use a ResNet-18 backbone to regress 22 hand joint angles. Experiments show that naive multi-view fusion does not always outperform single-view performance, motivating the need for more advanced multi-view reasoning methods.

1 Introduction

Ego-centric hand pose estimation is an important problem in augmented and virtual reality applications. The HoT3D dataset provides synchronized multi-view recordings from wearable devices along with ground-truth hand pose annotations. This project focuses on building baseline models for predicting hand joint angles from image data.

The goal of Phase 1 is to establish reference performance using simple methods before developing more advanced multi-view fusion techniques. Specifically, we implement and evaluate single-view and naive multi-view baselines.

2 Dataset and Pipeline

The HoT3D dataset contains multi-view video recordings along with synchronized hand pose annotations. Each recording includes images from multiple cameras and corresponding ground-truth hand joint angles.

A custom PyTorch dataset loader was implemented to:

- Parse hand pose annotations from JSONL files
- Load synchronized image frames from VRS recordings
- Align images with ground-truth timestamps
- Return samples in both single-view and multi-view formats

Images are converted to grayscale tensors and resized to 224×224. The dataset supports two modes:

- **Single-view:** one camera image per sample
- **Multi-view:** left and right camera images per sample

3 Models

All models are based on a ResNet-18 backbone adapted for grayscale input. The final fully connected layer is modified to output 22 hand joint angles.

3.1 Single-View Baseline

A single image is passed through the network to directly predict joint angles.

3.2 Multi-View Average Fusion

Predictions from two camera views are computed independently and averaged:

$$\hat{y} = \frac{1}{2}(\hat{y}_{left} + \hat{y}_{right})$$

3.3 Multi-View Concatenation

Features from both views are extracted, concatenated, and passed through a linear layer to produce the final prediction.

4 Training Setup

All models are trained using the Adam optimizer with a learning rate of 1×10^{-3} and mean squared error (MSE) loss. The dataset is split into 80% training and 20% validation sets.

Training is performed for 10 epochs with a batch size of 4. The best validation loss is recorded for each model.

5 Results

Table 1: Baseline performance on validation set

Model	Best Val Loss
Single-view	0.0383
Multi-view Average	0.0394
Multi-view Concatenation	0.0391

6 Analysis

The single-view baseline achieves strong performance, suggesting that a single camera contains sufficient information for predicting hand pose in many cases. Interestingly, the naive multi-view average model performs slightly worse than the single-view model.

The multi-view concatenation model improves over naive averaging by learning feature-level fusion between the two camera views. However, it still performs slightly worse than the single-view baseline overall.

These results suggest that simply adding additional views does not automatically improve performance. One camera view may contain occlusions, motion blur, or less informative features, which can negatively affect naive fusion methods. More advanced fusion strategies that account for visibility and view quality may be necessary to fully benefit from multi-view information.